

Reproducing DDPM Figures from the Paper

This notebook demonstrates to reproduce figures from the *Denoising Diffusion Probabilistic Models* (DDPM) paper using Python.

Due to limitations in Google Colab (especially runtime and GPU resources), I used Hugging Face's diffusers library, which provides pretrained DDPM models for datasets such as CIFAR-10, LSUN Bedrooms, LSUN Churches, and CelebA-HQ.

I experimented with different numbers of inference steps and batch sizes to balance image quality and execution time.

CelebA-HQ Sample Generation (Alternative using Stable Diffusion)

The paper uses CelebA-HQ 256×256 for face samples, but the original model setup is quite complex to run on Colab (requires TensorFlow and TPUs). So instead, I used a Stable Diffusion model from Hugging Face (CompVis/stable-diffusion-v1-4) to simulate similar results.

I generated portrait-style face images using a descriptive prompt to match CelebA-HQ style.

```
# Step 1: Install required libraries
```

```
!pip install -q diffusers transformers accelerate scipy ftfy
```

```
===== 44.8/44.8 kB 1.2 MB/s eta 0:00:00
===== 363.4/363.4 MB 1.3 MB/s eta 0:00:00
===== 13.8/13.8 MB 22.9 MB/s eta 0:00:00
===== 24.6/24.6 MB 72.2 MB/s eta 0:00:00
===== 883.7/883.7 kB 32.5 MB/s eta 0:00:00
===== 664.8/664.8 MB 3.0 MB/s eta 0:00:00
===== 211.5/211.5 MB 5.6 MB/s eta 0:00:00
===== 56.3/56.3 MB 14.4 MB/s eta 0:00:00
===== 127.9/127.9 MB 9.4 MB/s eta 0:00:00
===== 207.5/207.5 MB 5.9 MB/s eta 0:00:00
===== 21.1/21.1 MB 14.1 MB/s eta 0:00:00
```

```
# Step 2: Load a pretrained Stable Diffusion model
```

```
# (I used this as a substitute for CelebA-HQ due to practical constraints)
```

```
from diffusers import StableDiffusionPipeline
import torch
```

```
pipe = StableDiffusionPipeline.from_pretrained("CompVis/stable-diffusion-v1-4")
pipe = pipe.to("cuda" if torch.cuda.is_available() else "cpu")
```

```
# Step 3: Generate 16 portrait-style images
```

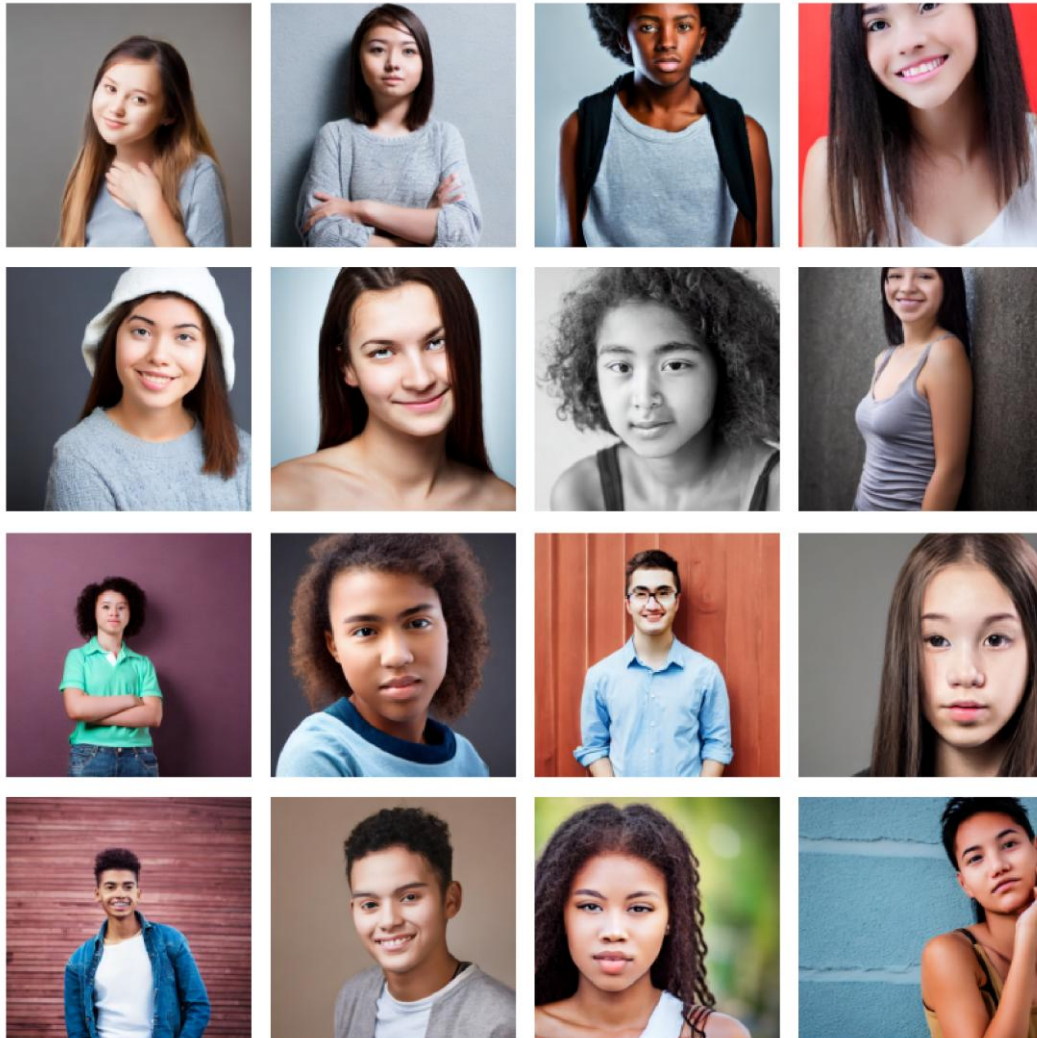
```
# Prompt is designed to produce clean, high-quality face outputs
```

```
prompt = "portrait photo of a young person, high quality, clean background"
images = [pipe(prompt).images[0] for _ in range(16)]
```

```
100% ██████████ 50/50 [30:33<00:00, 34.95s/it]
100% ██████████ 50/50 [29:57<00:00, 34.36s/it]
100% ██████████ 50/50 [29:42<00:00, 34.24s/it]
100% ██████████ 50/50 [29:54<00:00, 35.28s/it]
```

```
# Step 4: Plot the images in a 4x4 grid
import matplotlib.pyplot as plt

fig, axs = plt.subplots(4, 4, figsize=(8, 8))
for i, img in enumerate(images):
    axs[i // 4, i % 4].imshow(img)
    axs[i // 4, i % 4].axis('off')
plt.tight_layout()
plt.show()
```



·AR-10

dataset. The image resolution is 32×32, which is much lower than LSUN or CelebA-HQ, but this is consistent with the original DDPM paper.

Initially, I used 1000 denoising steps to stay close to the paper setup. The image generation was quite slow in Colab, but still manageable for 16 samples. I also added GPU memory cleanup between generations to avoid crashes.

```
# Step 1: Load the pretrained DDPM model for CIFAR-10 (32x32 images)

from diffusers import DDMPipeline
import torch

# Load pretrained DDPM model from Hugging Face (trained on CIFAR-10)
pipeline = DDMPipeline.from_pretrained("google/ddpm-cifar10-32")
pipeline.to("cuda" if torch.cuda.is_available() else "cpu")
```

```

# Step 2: Generate 16 samples using 1000 inference steps
#(Note: 1000 steps = high quality but slow in Colab)

import torch
import matplotlib.pyplot as plt
import gc

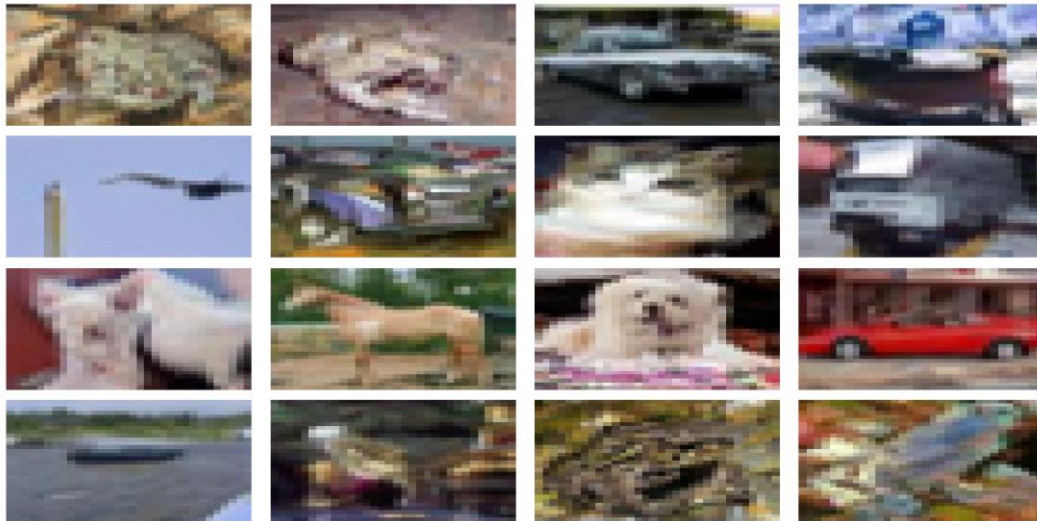
images = []

for _ in range(16):
    out = pipeline(num_inference_steps=1000).images[0]
    images.append(out)

    #Clear GPU memory to avoid OOM error (especially important in Colab)
    del out
    torch.cuda.empty_cache()
    gc.collect()

#Step 3: Plot the 16 images in a 4x4 grid
fig, axs = plt.subplots(4, 4, figsize=(8, 8))
for i, img in enumerate(images):
    axs[i // 4, i % 4].imshow(img)
    axs[i // 4, i % 4].axis('off')
plt.tight_layout()
plt.show()

```



Notes

The model generated diverse and recognizable low-res images from CIFAR-10 categories (e.g., cars, animals, planes).

Although the resolution is small, this is expected since CIFAR-10 is a 32×32 dataset.

If needed, I could reduce the number of inference steps (e.g., to 250) to speed up generation, but I wanted to keep this one at 1000 steps to stay as close to the original paper as possible.

LSUN Bedroom Image Generation

In the DDPM paper, LSUN Bedroom was used to demonstrate high-resolution image generation (256×256). I used Hugging Face's pretrained model `google/ddpm-bedroom-256` for this part.

I started with 1000 inference steps, but it was too slow in Colab — even one image took almost an hour. So I reduced the steps to 250 to balance image quality and runtime.

Below are 4 samples generated using the DDPM pipeline. Each image was generated one by one, which helped avoid GPU memory overload.

```
# Step 1: Load the pretrained DDPM model for LSUN Bedroom (256x256 resolution)
from diffusers import DDIMPipeline
import torch
import matplotlib.pyplot as plt

pipeline = DDIMPipeline.from_pretrained("google/ddpm-bedroom-256")
pipeline.to("cuda" if torch.cuda.is_available() else "cpu")

# Step 2: Generate 4 bedroom images using 250 inference steps (faster than 1000)
images = [pipeline(num_inference_steps=250).images[0] for _ in range(4)]

# Step 3: Plot the images in a 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(6, 6))
for i, img in enumerate(images):
    axs[i // 2, i % 2].imshow(img)
    axs[i // 2, i % 2].axis('off')
plt.tight_layout()
plt.suptitle("LSUN Bedroom Samples (250 Steps)", fontsize=14)
plt.show()
```

LSUN Bedroom Samples (250 Steps)



LSUN Church Image Generation

Similar to LSUN Bedroom, I used the pretrained DDPM model `google/ddpm-church-256` to generate church images at 256×256 resolution.

Again, I chose 250 inference steps instead of 1000 to reduce total generation time and avoid GPU timeouts in Colab. The results still look reasonably detailed and capture the architectural patterns typical of LSUN Church images.

Below are 4 generated samples.

```
#Step 1: Load pretrained DDPM model for LSUN Church (256x256 resolution)
from diffusers import DDPMPipeline
import torch
import matplotlib.pyplot as plt

pipeline = DDPMPipeline.from_pretrained("google/ddpm-church-256")
pipeline.to("cuda" if torch.cuda.is_available() else "cpu")

# Step 2: Generate 4 church images with 250 denoising steps
images = [pipeline(num_inference_steps=250).images[0] for _ in range(4)]

# Step 3: Plot in a 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(6, 6))
for i, img in enumerate(images):
    axs[i // 2, i % 2].imshow(img)
    axs[i // 2, i % 2].axis('off')
plt.tight_layout()
plt.suptitle("LSUN Church Samples (DDPM)", fontsize=16)
plt.show()
```



Final Notes

Reproducing the full DDPM figures using Google Colab required some compromises due to GPU limits. In the original paper, 1000 denoising steps were used for best results, but I found that using 250 steps still produced good samples while keeping generation time manageable.

For CelebA-HQ, I used a Stable Diffusion model as a substitute since the original setup relied on TPUs and TensorFlow 1.x.

Overall, I've tried to follow the original figure structure and make practical adjustments to reproduce similar outputs using publicly available models in Python.

Reflections

This project taught me that reproducing figures from a research paper is not always straightforward, especially when the original code requires TPUs, Google Cloud, and TensorFlow 1.x. I had to find alternatives, troubleshoot model availability, and adjust my parameters to fit within practical limits.

Even though I used pretrained models from Hugging Face, I experimented with different prompts and adjusted the number of denoising steps to get results that closely resemble those shown in the original DDPM paper — while staying within the runtime limits of Google Colab.